

Student Name: Ya Zou

Email: zouy2@oregonstate.edu

Project Number: #1

Project Name: Monte Carlo Simulation

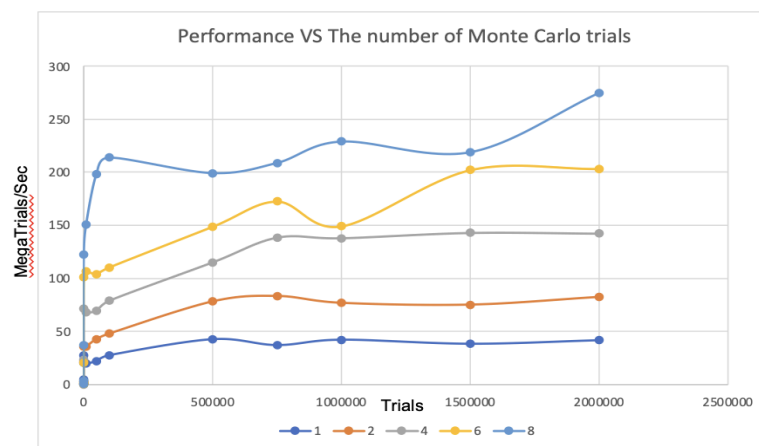
Project Report #1

1. A rectangular data table of the performance numbers as a function of NUMT and NUMTRIALS.

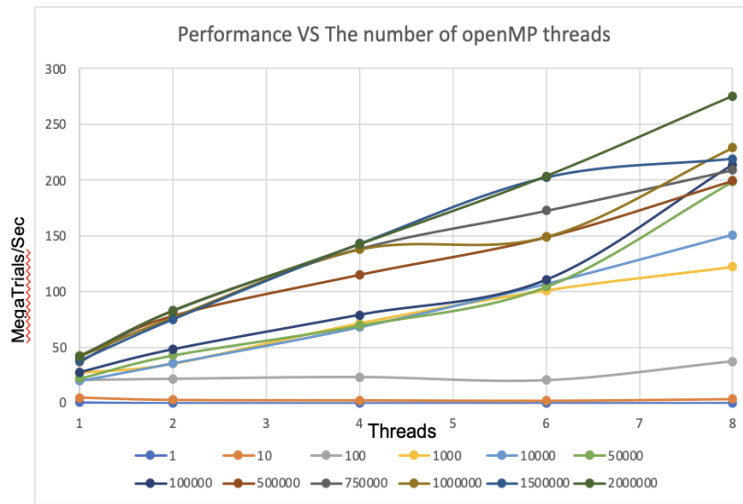
NUMTRIALS/NUMT	1	2	4	6	8
1	0.6	0.3	0.24	0.25	0.34
10	4.81	2.99	2.63	2.28	3.56
100	20.8	21.86	23.38	20.67	37.13
1000	27.1	35.41	71.55	101.11	122.14
10000	19.91	35.76	68.03	106.68	150.52
50000	22.23	42.91	69.57	104.18	198.28
100000	27.53	48.01	78.96	110.4	213.75
500000	42.67	78.22	115.04	148.65	199.23
750000	37.1	83.35	138.38	172.43	208.79
1000000	42.19	76.89	137.84	149.04	228.88
1500000	38.34	75.22	142.8	202.12	218.91
2000000	41.7	82.47	142.16	203.23	274.96

2. The 2 performance graphs. The two graphs need to be:
 - a. Performance versus the number of Monte Carlo trials, with the colored lines being the number of OpenMP threads.
 - b. Performance versus the number of OpenMP threads, with the colored lines being the number of Monte Carlo trials.

a.



b.



3. Your estimate of the Probability.

Probability = 57.01

4. Your estimate of the Parallel Fraction (*show your work!*).

$$Speedup_n = \frac{T_1}{T_n} = \frac{P_n}{P_1}$$

$$F = \frac{\frac{1}{S} - 1}{\frac{1-n}{n}} = \frac{n}{n-1} \cdot \frac{Speedup - 1}{Speedup}$$

$$Speedup = p_n/p_1 = 274.96 / 41.70 = 6.594$$

$$F_{parallel} = 8/8-1 * (6.594 - 1/6.594) = 1.142857 * 0.84835 = 0.969542$$

Therefore, My parallel Fraction is 0.969542

5. **Commentary:** why do the graphs look the way they do? What are they telling you?

The graphs illustrate how the program's performance changes with varying dataset sizes. Initially, for smaller datasets (0 to 500,000), thread initialization overhead dominates, leading to poorer performance. As the dataset grows (500,000 to 2,000,000), performance improves due to the increased computational workload. However, slight curves in the graphs suggest external factors, like system interruptions, affecting performance intermittently. As mentioned by the professor, it's possible that during execution, interruptions occur where other code is swapped into the system, impacting the program's performance temporarily. These fluctuations introduce variability in the execution time, leading to the observed curves in the line graphs.

The graphs show us how parallel programming works and how Amdahl's Law affects it. At first, when we make things parallel, the program runs faster as we use more computers or processors. But the graphs also show that this speedup doesn't keep getting better and better as we add more computers. This happens because some parts of the program have to be done one after the other, and we can't speed them up by making them parallel. These parts act like a roadblock to making the whole program faster. The wavy lines in the graphs show that sometimes other things happening on the computer can slow down our program too. So, the graphs teach us that we have to be smart about using parallel programming and not expect it to make everything super fast all the time. We have to balance how much we make parallel and how much we do one thing at a time to get the best performance.